

## Experiment 6

# Design and Implementation of a 4-bit Common Bus System using Multiplexers and Registers

|                |           |         |             |
|----------------|-----------|---------|-------------|
| Experiment No. | 6         | Subject | COA         |
| Branch         | CSE       | Name    | Anshul Modi |
| Roll No.       | 241210021 |         |             |

### 1. Aim / Objective

To design and implement a **4-bit Common Bus System** using **multiplexers** and **registers** on Logisim. The system allows any one of four 4-bit registers (R0–R3) to place its data onto a shared common bus, selected by a 2-bit select signal.

### 2. Theory

A **common bus system** is a fundamental building block in computer architecture. It provides a shared communication pathway through which multiple registers can transfer data to other components. At any given time, only one register is allowed to place data on the bus, while others remain disconnected. This is controlled by a **multiplexer (MUX)**.

#### Key Components:

- **Registers (R0–R3):** Each is a 4-bit D-type register with a Write Enable (WE) input. Data is loaded into the register only when  $WE = 1$  and a clock edge occurs.
- **Multiplexer (MUX):** A  $4 \times 1$  MUX with a 2-bit select input (S1, S0) routes the output of the selected register onto the common bus.
- **Common Bus:** A 4-bit shared line that carries data from the selected register to any destination component (e.g., another register or ALU).
- **Clock:** Synchronises the loading of data into registers.

The MUX select lines determine which register drives the bus. When a register's output is selected, its 4-bit data appears on the bus. The receiving register then loads this data when its WE line is asserted.

### 3. Register Configuration

The four 4-bit registers are pre-loaded with the following input values as shown in the circuit:

| Register | 4-bit Input (shown in circuit) | WE (Write Enable) | Function                |
|----------|--------------------------------|-------------------|-------------------------|
| R0       | 1010                           | 1                 | Stores data when $WE=1$ |
| R1       | 0101                           | 1                 | Stores data when $WE=1$ |
| R2       | 0100                           | 1                 | Stores data when $WE=1$ |
| R3       | 0000                           | 1                 | Stores data when $WE=1$ |

#### 4. Bus Selection Truth Table

The 2-bit select input to the MUX determines which register's data appears on the common bus:

| S1 (Select) | S0 (Select) | Bus Output (Register on Bus) |
|-------------|-------------|------------------------------|
| 0           | 0           | R0 placed on bus             |
| 0           | 1           | R1 placed on bus             |
| 1           | 0           | R2 placed on bus             |
| 1           | 1           | R3 placed on bus             |

#### 5. Procedure

1. Open Logisim and create a new project.
2. Add four 4-bit registers (R0, R1, R2, R3) from the Memory library.
3. Connect 4-bit input pins to the Data (D) input of each register.
4. Connect Write Enable (WE) pins to each register to control data loading.
5. Connect a clock signal to all four registers.
6. Add a 4×1 Multiplexer (or cascade 2×1 MUXes) to select among the four register outputs.
7. Connect the Q outputs of R0, R1, R2, R3 to the four inputs of the MUX.
8. Add a 2-bit select input (S1, S0) to the MUX select lines.
9. Connect the MUX output to the common bus (a 4-bit wire).
10. Attach a 4-bit output display (e.g., LED/probe) to the bus to observe the selected data.
11. Load data into registers by setting WE = 1 and toggling the clock.
12. Change the select lines (S1, S0) to verify that the correct register data appears on the bus.
13. Record observations for all four select combinations.

#### 6. Logisim Circuit Screenshot

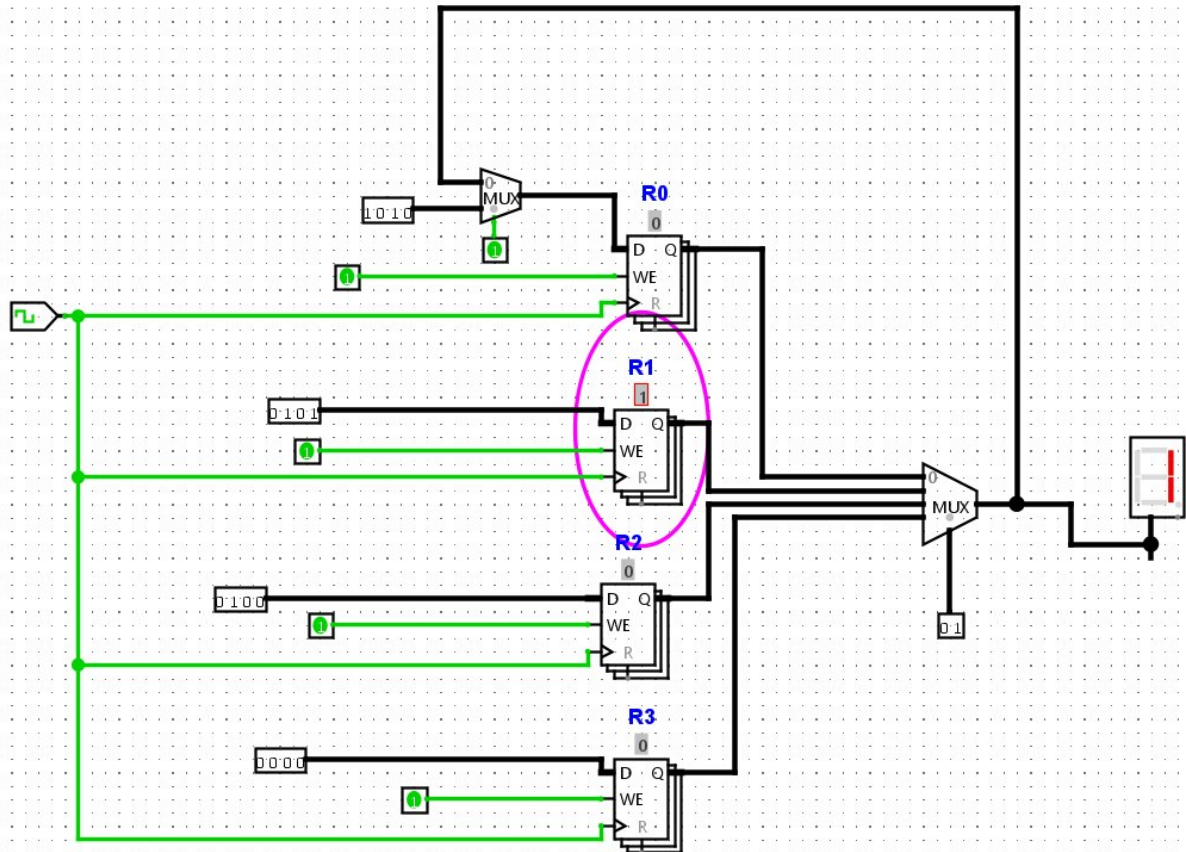


Fig. 1 – 4-bit Common Bus System on Logisim: Registers R0–R3 with MUX-based bus selection. R1 is currently selected (circled in pink), placing its data (0101) on the common bus.

## 7. Observations

- When select lines  $S1=0$ ,  $S0=0$ , the output of **R0 (1010)** was placed on the common bus.
- When select lines  $S1=0$ ,  $S0=1$ , the output of **R1 (0101)** was placed on the common bus.
- When select lines  $S1=1$ ,  $S0=0$ , the output of **R2 (0100)** was placed on the common bus.
- When select lines  $S1=1$ ,  $S0=1$ , the output of **R3 (0000)** was placed on the common bus.
- Only the selected register drove the bus at any given time; others remained isolated.
- The Write Enable (WE) signal correctly controlled data loading into each register.
- The system correctly demonstrated the shared bus architecture used in real CPU datapaths.

## 8. Result

A 4-bit Common Bus System was successfully designed and implemented on Logisim using registers and a multiplexer. The MUX correctly routed the selected register's 4-bit data onto the shared bus for all four select combinations, confirming the correct operation of the common bus architecture.